Introduction to Web Technologies

SWE230 :
WEB APPLICATION PROGRAMMING

Spring 2026

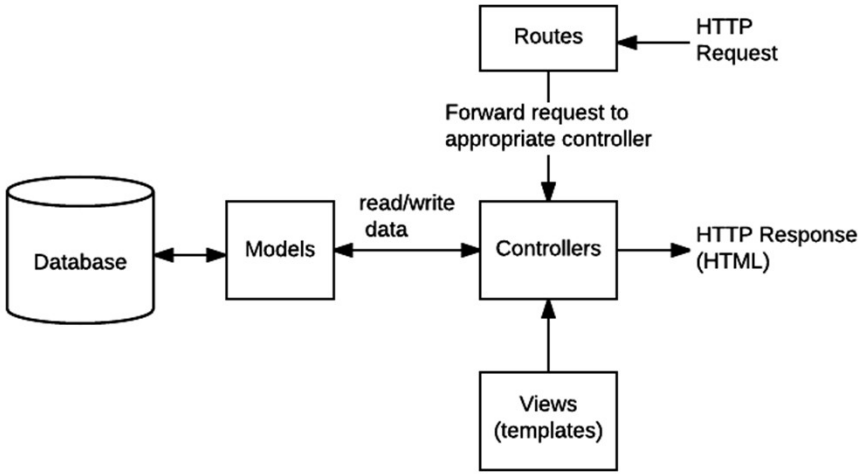
Instructors

- **Sarah Nabil**
  Sara.abdullah@miuegypt.edu.eg
- **Nada Ayman**
  nada.ayman@miuegypt.edu.eg
- **Nada AbdelFattah**
  nada.abdelfattah@miuegypt.edu.eg

MVC

What is MVC Architecture?

- MVC stands for Model-View-Controller.
  - Model: Handles data and database logic.
  - View: The user interface (EJS templates).
  - Controller: Business logic and request handling.
- Helps organize code and separate concerns.



MVC Architecture

Project Structure

- project/
  - controllers/
  - models/
  - views/
  - routes/
  - public/
    - css/
    - js/
    - images/

NoSQL Databases

NoSQL (which stands for Not-only-SQL) is category of database software that describes a style of database that doesn't use the relational table model of normal SQL databases.

NoSQL databases rely on a different set of ideas for data modeling that put fast retrieval ahead of other considerations like consistency.

Systems like DynamoDB, Firebase, and MongoDB now power thousands of sites including household names like Netflix, eBay, Instagram, Forbes, Facebook, and others.

Why (and Why Not) Choose NoSQL?

NoSQL systems handle huge datasets better than relational systems.

NoSQL databases aren't the best answer for all scenarios. SQL databases use schemas for a very good reason: they ensure data consistency and data integrity.

The data in most NoSQL database systems is identified by a unique key. The key-value organization often results in faster retrieval of data in comparison to a relational database

Key-Value Stores

Key-value stores alone are quite straightforward in that every value, whether an integer, string, or other data structure, has an associated key (i.e., they are analogous to PHP associative arrays)

Here every value has a key. This allows fast retrieval through means such as a hash function, and precludes the need for indexes on multiple fields as is the case with SQL

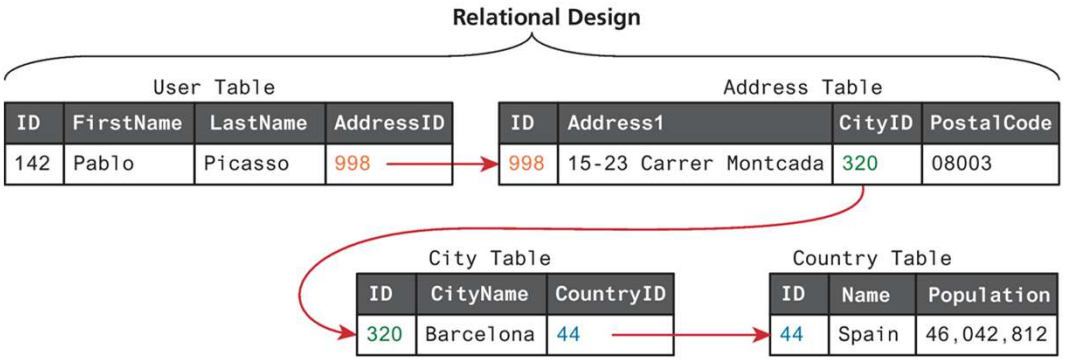
Key	Value
Customer.Name	"Randy"
Price	200.00
ShippingAddress	"4825 Mount Royal Gate SW"
Countries	"Canada","France","Germany","United States"

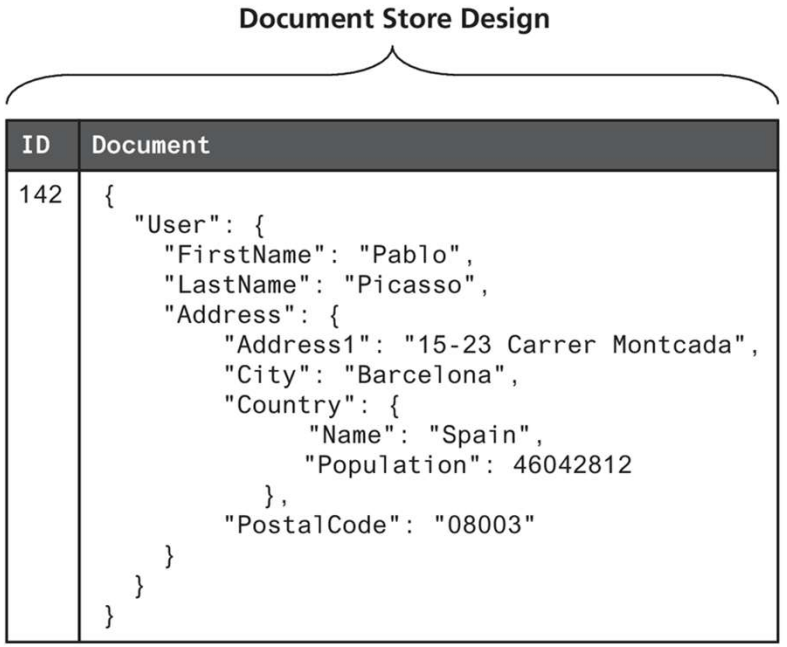
Document Store

Document Stores (also called document-oriented databases) associate keys with values, but unlike key-value stores, they call that value a document.

- A document can be a binary file like a .doc or .pdf or a semi-structured XML or JSON document.
- Most NoSQL systems are of this type. MongoDB, AWS DynamoDB, Google FireBase, and Cloud Datastore are popular examples.

Relational data versus document store data

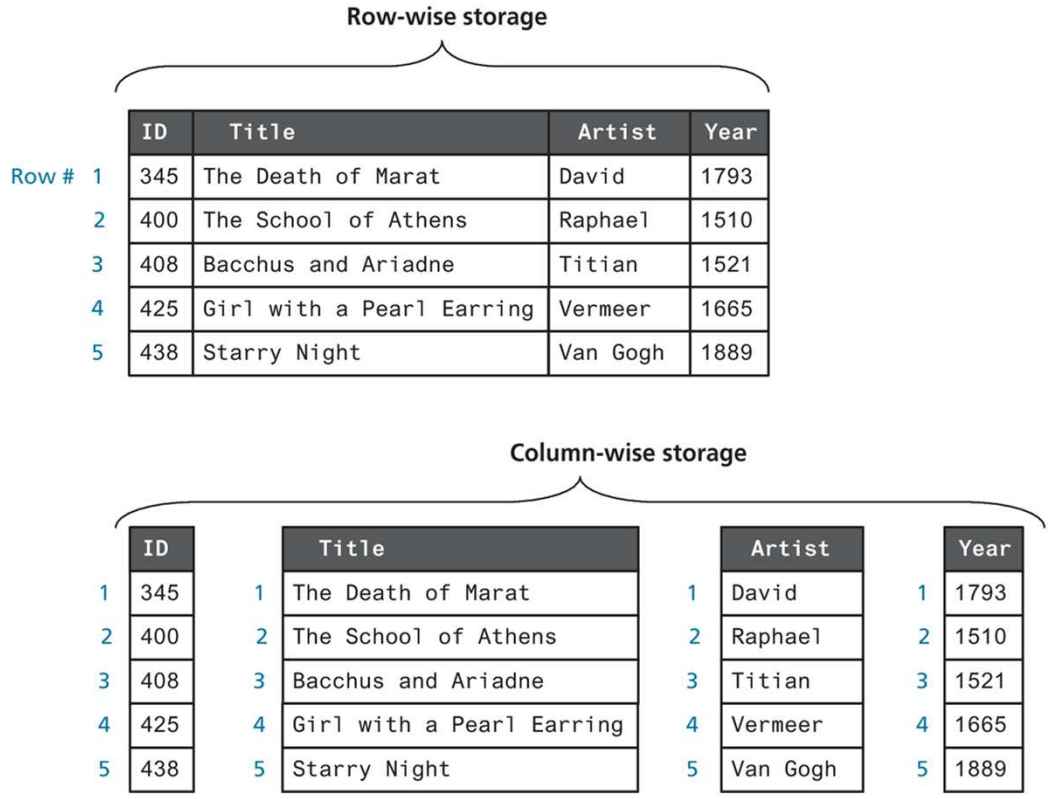




Column Stores

In traditional relational database systems, the data in tables is stored in a row-wise manner. This means that the fundamental unit of data retrieved is a row.

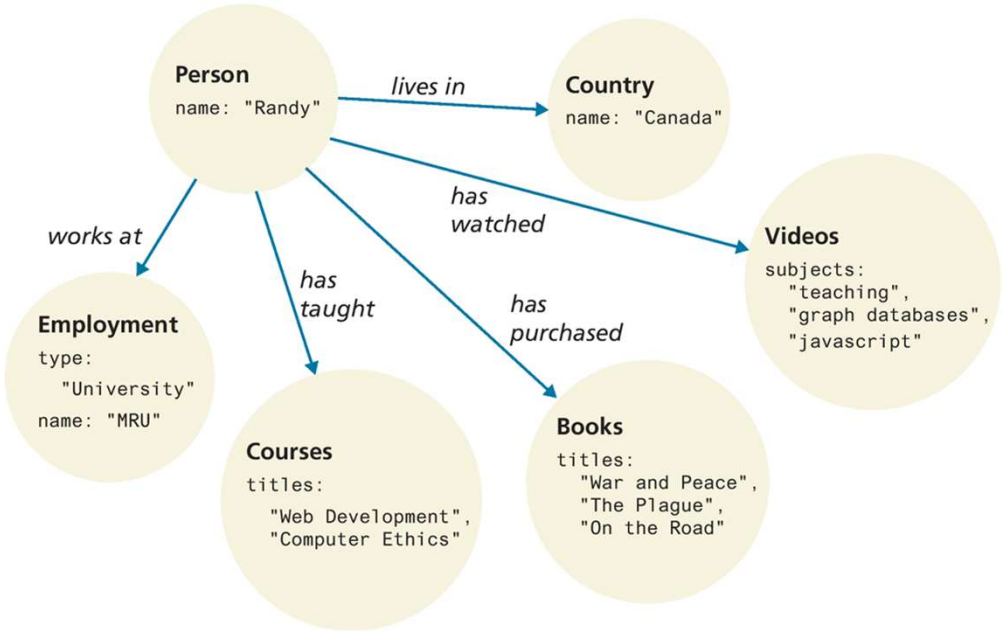
Column Store systems store data by column instead of by row



Graph Stores

In a **Graph Store** system (often simply called graph databases), data is represented as a network or graph of entities and their relationships.

Some examples of graph databases include Neo4j, OrientDB, and RedisGraph.



Working with MongoDB in Node

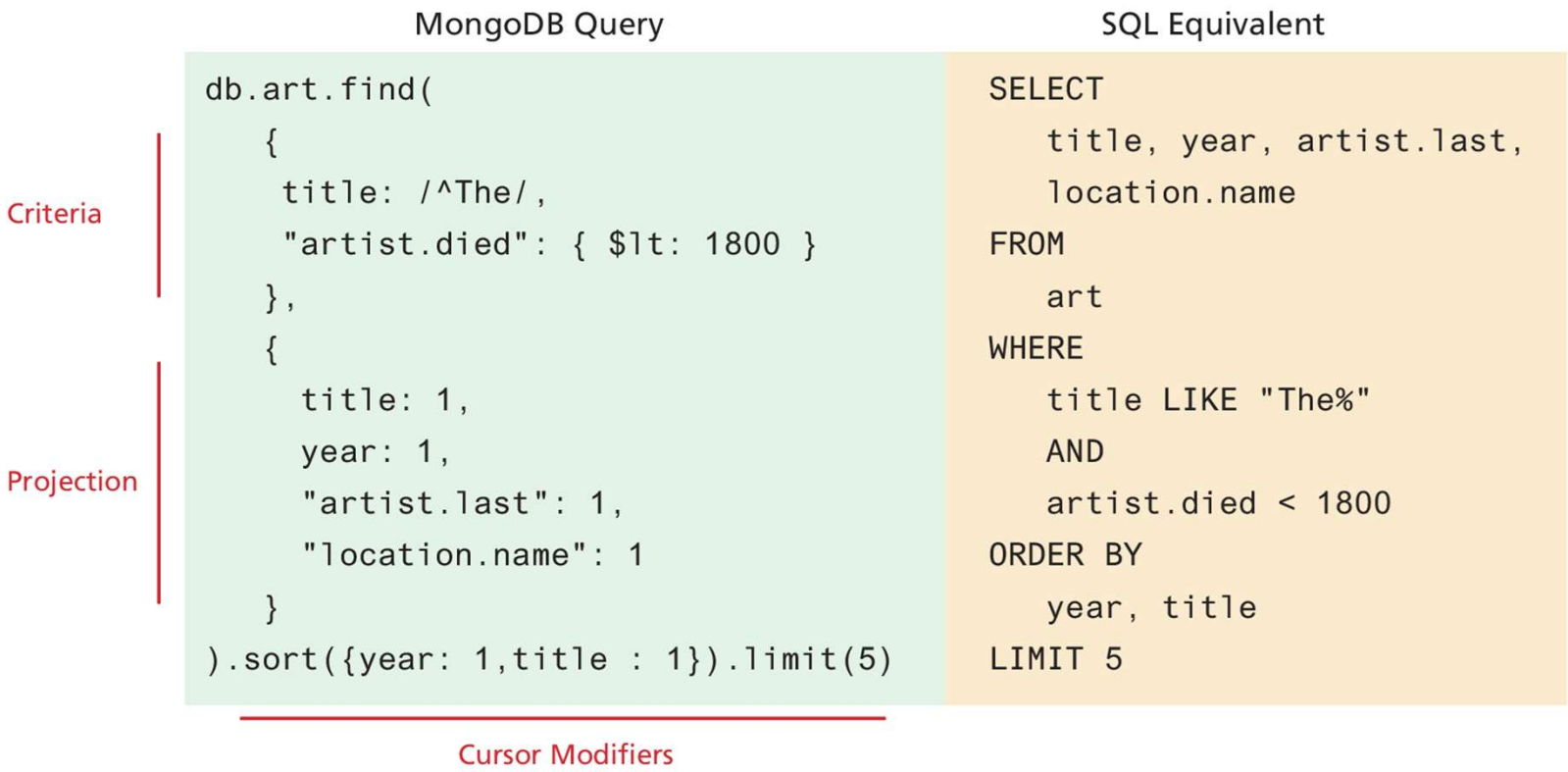
- MongoDB MongoDB is an open-source, NoSQL, document-oriented database. It can be used with Nodejs, it is much more commonly used with Node
- You simply package your data as a JSON object, give it to MongoDB, and it stores this object or document as a binary JavaScript object (BSON).
- MongoDB does not support transactions
- The ability to run on multiple servers means MongoDB can handle large datasets

Comparing relational databases to the MongoDB data model

RDBMS		MongoDB
Database	⇨	Database
Table, View	⇨ ⇨	Collection
Row	⇨ ⇨	Document (JSON, BSON)
Column		Field
Index	⇨	Index
Join	⇨	Embedded Document
Foreign Key	⇨	Reference

Comparing relational databases to the MongoDB data model

Comparing a MongoDB query to an SQL query



Accessing MongoDB Data in Node.js

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in database* given its schema
3. Apply CRUD operations

The `dotenv` package is used to load environment variables from a `.env` file into `process.env` through calling config function

- This is a common practice to keep sensitive information (like database URLs, API keys) out of the codebase.
- Example of .env file content
  MONGO_URL=mongodb+srv://user:password@cluster0.example.com/dbname?retryWrites=true&w=majority

```
require('dotenv').config();
console.log(process.env.MONGO_URL);

const mongoose = require('mongoose');
mongoose.connect(process.env.MONGO_URL) .then(()
=> {
console.log('Connected to MongoDB');
}) .catch(err => {
  console.error('Initial connection error:', err);
});
```

Accessing MongoDB Data in Node.js

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in the database* given its schema
3. Apply CRUD operations

```
// define a schema that maps to the structure in MongoDB
const bookSchema = new mongoose.Schema({
        id: Number,
        isbn10: String,
        isbn13: String,
        title: String,
        …
        ,
        category: {
                main: String,
                secondary: String
        }
});
// now create model using this schema that maps to books collection in database
module.exports = mongoose.model('Book', bookSchema,'books');
```

Accessing MongoDB Data in Node.js

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in the database* given its schema
3. Apply CRUD operations

1. Field Type Definition
The most basic parameter specifies the data type for a field:

```
javascript
const userSchema = new mongoose.Schema({
  name: String,       // Shorthand for { type: String }
  age: Number,        // Shorthand for { type: Number }
  isAdmin: Boolean,   // Shorthand for { type: Boolean }
  createdAt: Date     // Shorthand for { type: Date }
});
```

Accessing MongoDB Data in Node.js

Schema Options Object: For advanced configuration, use an object with these common options:

Option	Description	Example
type	Data type (String, Number, Date, Buffer, Boolean, Mixed, ObjectId, Array, etc.)	type: mongoose.Schema.Types.ObjectId
required	Field is mandatory	required: true
default	Default value if not provided	default: Date.now
unique	Enforces unique values in the collection	unique: true
min / max	Minimum/maximum values (for numbers/dates)	min: 18, max: 100
minlength / maxlength	Min/max character length (for strings)	minlength: 3, maxlength: 30
enum	Restrict to predefined values	enum: ['user', 'admin', 'moderator']
trim	Automatically trim whitespace (for strings)	trim: true
select	Control if field is included by default in queries	select: false (exclude by default)
validate	Custom validation function	validate: (v) => v.length > 0
ref	Reference to another model (for relationships)	ref: 'User'
immutable	Prevents field modification after creation	immutable: true
get / set	Transform value when getting/setting	get: v => Math.round(v)

Accessing MongoDB Data in Node.js: for Referenced documents

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in the database* given its schema
    1. Define Schemas with References
    2. Create schemas where the child model (Book) references the parent model (Author):
    3. Use populate() in queries to replace the author ID with the actual referenced document
3. Apply CRUD operations

```
// Author Model (parent)
const authorSchema = new mongoose.Schema({
  name: String,
});
const Author = mongoose.model('Author', authorSchema);

// Book Model (child)
const bookSchema = new mongoose.Schema({
  title: String,
  author: {
    type: mongoose.Schema.Types.ObjectId, // Reference ID
    ref: 'Author', // Model to reference
    required: true
  }
});
const Book = mongoose.model('Book', bookSchema);
```

Accessing MongoDB Data in Node.js: for Referenced documents

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in the database* given its schema
    1. Define Schemas with References
    2. Create schemas where the child model (Book) references the parent model (Author):
    3. Use populate() in queries to replace the author ID with the actual referenced document
3. Apply CRUD operations

2. Create Documents
Link documents by saving the parent's _id in the child's reference field:

```
// Create an author
const newAuthor = new Author({ name: 'J.K. Rowling' });
await newAuthor.save();

// Create a book linked to the author
const newBook = new Book({
  title: 'Harry Potter',
  author: newAuthor._id // Store the author's ID
});
await newBook.save();
```

Accessing MongoDB Data in Node.js: for Referenced documents

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in the database* given its schema
    1. Define Schemas with References
    2. Create schemas where the child model (Book) references the parent model (Author):
    3. Use populate() in queries to replace the author ID with the actual referenced document
3. Apply CRUD operations

3. Perform a Join with populate()
Use populate() in queries to replace the author ID with the actual author document:

```
javascript
// Find books and populate author details
const books = await Book.find().populate('author');
// Result:
// [
//   {
//     _id: '...',
//     title: 'Harry Potter',
//     author: { _id: '...', name: 'J.K. Rowling' }
//   }
// ]
```

Accessing MongoDB Data in Node.js

1. Connect to DB URL: which can be local or remote
2. Create a model *schema that maps to a collection in the database* given its schema
3. Apply CRUD operations

1. Field Type Definition
The most basic parameter specifies the data type for a field:

javascript

```
const userSchema = new mongoose.Schema({
  name: String,      // Shorthand for { type: String }
  age: Number,       // Shorthand for { type: Number }
  isAdmin: Boolean,  // Shorthand for { type: Boolean }
  createdAt: Date    // Shorthand for { type: Date }
});
```

CRUD Operations with Mongoose

- Create: `Model.create()` or `new Model().save()`
- Read: `Model.find()`, `Model.findById()`
- Update: `Model.findByIdAndUpdate()`
- Delete: `Model.findByIdAndRemove()`

CRUD Operations with Mongoose

- Create: `Model.create()` or `new Model().save()`
- Read: `Model.find()`, `Model.findById()`
- Update: `Model.findByIdAndUpdate()`
- Delete: `Model.findByIdAndRemove()`

```
//Create an Author:
const newAuthor = new Author({ name: 'George R.R. Martin' });
const savedAuthor = await newAuthor.save();
console.log('Created author:', savedAuthor);
Create a Book (with Author reference):
 const newBook = new Book({
  title: 'A Game of Thrones',
  author: savedAuthor._id // Reference author's ID
});
const savedBook = await newBook.save();
console.log('Created book:', savedBook);
```

CRUD Operations with Mongoose

- Create: `Model.create()` or `new Model().save()`
- Read: `Model.find()`, `Model.findById()`
- Update: `Model.findByIdAndUpdate()`
- Delete: `Model.findByIdAndRemove()`

```
//Get All Books with Author Details:
const books = await Book.find().populate('author’);
//Get Single Book with Author:
const book = await Book.findById(savedBook._id).populate('author’);
//Get All Books by Specific Author:
const authorBooks = await Book.find({ author: savedAuthor._id });
//Get Author with Their Books:
const author = await Author.findById(savedAuthor._id).populate('books'); // Populate book references
```

CRUD Operations with Mongoose

- Create: `Model.create()` or `new Model().save()`
- Read: `Model.find()`, `Model.findById()`
- Update: `Model.findByIdAndUpdate()`
- Delete: `Model.findByIdAndRemove()`

```
//Update Book Title:
const updatedBook = await Book.findByIdAndUpdate(
  savedBook._id,
  { title: 'A Clash of Kings' },
  { new: true } // Return updated document
);
//Update Author's Name:
const updatedAuthor = await Author.findByIdAndUpdate(
  savedAuthor._id,
  { name: 'GRRM' },
  { new: true }
);
```

CRUD Operations with Mongoose

- Create: `Model.create()` or `new Model().save()`
- Read: `Model.find()`, `Model.findById()`
- Update: `Model.findByIdAndUpdate()`
- Delete: `Model.findByIdAndRemove()`

```
//Change Book's Author:

// Create new author

const newAuthor2 = await new Author({ name: 'Brandon Sanderson' }).save();

// Reassign book's author

await Book.findByIdAndUpdate(

  savedBook._id, { author: newAuthor2._id }

);

// Update both authors' books arrays

await Author.findByIdAndUpdate(

  savedAuthor._id, { $pull: { books: savedBook._id } } // Remove from old author

);

await Author.findByIdAndUpdate(

  newAuthor2._id,  { $push: { books: savedBook._id } } // Add to new author);
```

CRUD Operations with Mongoose

- Create: `Model.create()` or `new Model().save()`
- Read: `Model.find()`, `Model.findById()`
- Update: `Model.findByIdAndUpdate()`
- Delete: `Model.findByIdAndRemove()`

```
//Delete a Book:

const deletedBook = await Book.findByIdAndRemove(savedBook._id);

// Remove reference from author

await Author.findByIdAndUpdate( savedAuthor._id,  { $pull: { books: deletedBook._id } });

// Delete author and all their books

await Book.deleteMany({ author: savedAuthor._id }); // Delete associated books

await Author.findByIdAndRemove(savedAuthor._id); // Delete author

Alternative: Delete Author but Keep Books

// Remove author reference from books

await Book.updateMany( { author: savedAuthor._id }, { $unset: { author: "" } }); // Or set to null

// Then delete author

await Author.findByIdAndDelete(savedAuthor._id);
```

Using .populate()

- Retrieves referenced documents automatically.
- Example: `Book.find().populate('author')`
- Useful for showing complete relational data in queries.

Thank you